OpenZeppelin | security

OpenZeppelin Accounts Social Recovery Module Audit

OpenZeppelin

October 8, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Social Recovery Mechanism 5

Guardian Authorization 5

Delayed Execution 6

Security Model and Trust Assumptions 6

Low Severity 8

L-01 Uninstalled Social Recovery Modules Allow Permissionless Cancellation 8

L-02 Recovery Signatures Do Not Bind the Consuming Module 9

L-03 ERC-5267 Domain Extensions Prevent Standard Guardian Recovery Signatures 10

Notes & Additional Information 12

N-01 Inaccurate Documentation of delayChangeWindow 12

Conclusion 13

Appendix 14

Issue Classification 14

Summary

Type	Library
Timeline	From 2026-09-21 To 2026-09-23
Languages	Solidity

Total Issues	4 (3 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	3 (2 resolved)
Notes & Additional Information	1 (1 resolved)
Client Reported Issues	0 (0 resolved)

Scope

We audited the OpenZeppelin Accounts repository at commit a044509.

In scope were the following files:

```
packages/contracts/src
├── modules
│   ├── ERC7579Multisig.sol
│   ├── ERC7579MultisigWeighted.sol
│   └── executors
│       ├── ERC7579DelayedExecutor.sol
│       ├── ERC7579Executor.sol
│       └── ERC7579SocialRecoveryExecutor.sol
└── utils
    └── Multisig.sol
```

`ERC7579SocialRecoveryExecutor.sol` and `Multisig.sol` were reviewed in full, while the remaining four files were reviewed as a diff against commit 6e14ef8, at which they were previously audited.

Update: *All resolutions and the final state of the audited codebase mentioned in this report are contained at commit 4ceb836.*

Please note that changes introduced to contracts not in-scope for this audit were not part of the additional reviews.

System Overview

The scope provides a social recovery module for ERC-7579 modular smart accounts. The module combines a weighted guardian quorum with delayed execution: guardians authorize an account operation, the operation remains pending for a configurable delay, and the account can cancel it before execution.

Social Recovery Mechanism

`ERC7579SocialRecoveryExecutor` composes `ERC7579DelayedExecutor` and `ERC7579MultisigWeighted`. During installation, the account configures its execution delay and expiration together with its guardian set, weights, and threshold.

To start a recovery, guardians sign an EIP-712 authorization over the operation's salt, execution mode, and calldata using the account's ERC-5267 domain. Anyone may submit the signatures through `startRecovery`. Once the configured delay has elapsed, `executeRecovery` is permissionless and forwards the signed ERC-7579 execution to the account. A scheduled recovery may be canceled either by the account or by a valid guardian quorum.

Guardian Authorization

`ERC7579Multisig` stores an independent signer set and threshold for each account. `ERC7579MultisigWeighted` assigns a weight to each signer and accepts an authorization when the participating signers reach the account's configured weight threshold. Signatures are checked using ERC-7913-compatible signer encodings, allowing guardians to be represented by EOAs or contract-based verifiers.

The `Multisig` library decodes the signer and signature arrays directly from calldata. The recovery module uses this decoder before checking that each signer is authorized, appears only once, and supplied a valid signature.

Delayed Execution

`ERC7579DelayedExecutor` tracks each operation through the `Unknown`, `Scheduled`, `Ready`, `Executed`, `Expired`, and `Canceled` states. Each account configures its own execution delay and expiration window. Delay decreases take effect only after the difference between the old and new delays has elapsed, while increases are bounded by `delayChangeWindow`.

The base `ERC7579Executor` dispatches ready operations through the account's `executeFromExecutor` entry point. Scheduled operations remain in module storage when an account uninstalls the module and may become actionable again after reinstallation if they have not expired or been canceled.

Security Model and Trust Assumptions

The following assumptions affect the security of the social recovery module:

- **Account enforcement:** The account is expected to restrict module installation, uninstallation, and `executeFromExecutor` access correctly. Configuration functions rely on `msg.sender` being the account whose recovery policy is being changed.
- **Guardian authority:** A guardian quorum can authorize arbitrary ERC-7579 execution calldata supported by the account. Guardians are therefore trusted collectively with the authority granted to the recovery module, including any account-management actions that the account permits.
- **Guardian configuration:** Guardian identifiers are assumed to use canonical ERC-7913 encodings and to represent distinct, controlled signers. Thresholds and weights must be configured consistently, and the signer set must remain within practical gas limits.
- **Typed-data domain:** The account is expected to implement ERC-5267 and accurately report the EIP-712 domain used for guardian signatures. Integrations must account for any domain extensions that the module does not support.
- **Monitoring and cancellation:** The delay protects the account only if pending recoveries are monitored and unauthorized operations can be canceled before they become executable. Anyone may execute an operation after it enters the `Ready` state.

- **Installation lifecycle:** Uninstalling the module does not clear scheduled operations. Accounts are expected to cancel unwanted recoveries before uninstalling or replacing the guardian configuration when those operations must not survive a later reinstallation.
- **Input encoding:** Installation data and multisignatures must follow the encodings expected by the delayed executor, weighted multisig, and `Multisig` decoder.

The in-scope contracts contain no global administrator or owner role. Privileged actions are performed by each account or by the guardian quorum configured for that account. The security of downstream deployments also depends on how the account grants executor permissions and which recovery calldata it allows the module to execute.

Low Severity

L-01 Uninstalled Social Recovery Modules Allow Permissionless Cancellation

The `onUninstall` function of `ERC7579SocialRecoveryExecutor` retains pending recovery operations while clearing the account's guardian configuration. In particular, `ERC7579Multisig.onUninstall` removes all signers and resets the threshold to zero, while the delayed executor intentionally preserves scheduled operations. However, `cancelRecovery` remains callable while the module is uninstalled and authorizes any non-account caller whose multisignature passes validation.

The `tryDecodeMultisignatureCalldata` function accepts 64 zero bytes as a valid encoding of two empty arrays. After uninstallation, these empty arrays pass multisignature validation: signature validation performs no iterations, while the weighted threshold check compares an accumulated weight of zero against the reset threshold of zero. Consequently, any caller can invoke `cancelRecovery` with the operation parameters and an empty multisignature to permanently cancel a retained `Scheduled` or `Ready` recovery. The required operation parameters are publicly available from the `ERC7579ExecutorOperationScheduled` event.

An attacker can therefore invalidate a pending recovery during a temporary uninstallation or guardian-rotation flow. Reinstalling the module does not restore the canceled operation, and the same operation identifier cannot be scheduled again. The guardians can authorize an equivalent recovery using a fresh salt after reinstallation, but doing so requires collecting new signatures and waiting through another complete recovery delay.

Consider preventing guardian-authorized cancellation while the recovery configuration is inactive, for example by requiring a nonzero threshold before validating a non-account caller's multisignature. Preserve the direct account-authorized cancellation path so that an account can still cancel retained operations while the module is uninstalled.

Update: *Resolved in pull request #94. The OpenZeppelin Accounts team stated:*

> *Resolved in PR #94.*

> *We took the suggested approach and now require a nonzero threshold before a multisignature can authorize the cancellation:*
>
> ```
> valid = success && threshold(account) > 0 && _validateMultisig(account,
> cancelTypedHash, signers, signatures);
> ```
>
> *The threshold is only ever zero when no guardian configuration is active, since `_setThreshold` rejects zero and is its only writer, so no in-use flow changes. The `msg.sender == account` path is untouched, so an account can still cancel retained operations while the module is uninstalled, as the finding recommends.*
>
> *We placed the guard in `cancelRecovery` rather than in `ERC7579Multisig._validateMultisig`. `_validateMultisig` composes the threshold check from the overridable `_validateThreshold` hook by design, and we did not want to fold an installation concern into it. It is also worth noting that `ERC7579MultisigWeighted._validateThreshold` and `ERC7579MultisigConfidential._validateMultisig` both override without calling `super`, so a guard in the base would not have reached every path regardless.*
>
> *`testUninstalledModuleRejectsGuardianCancel` covers both empty-multisignature encodings, the 64-zero-byte minimal form that `Multisig.tryDecodeMultisignatureCalldata` accepts and the standard `abi.encode` of two empty arrays, then confirms the operation is still `Scheduled` and that the account itself can still cancel it. We verified the test fails without the change.*

L-02 Recovery Signatures Do Not Bind the Consuming Module

The `SCHEDULE_RECOVER_TYPEHASH` includes the operation's `salt`, execution `mode`, and `recoveryCalldata`, but not the recovery module's address. Because the digest uses the account's EIP-712 domain, the same authorization is valid in compatible module deployments installed on that account. Each deployment also maintains independent schedule storage.

If an authorization is disclosed through one module and the recovery is later canceled, the same signature can be submitted to another module with an equivalent guardian configuration. The second module treats the operation as unscheduled and creates a new recovery that is unaffected by the first module's cancellation. This requires the account to install another compatible module and remains subject to that module's guardian policy and delay.

Consider including `address(this)` in the signed recovery and cancellation structures. An account-controlled nonce or epoch could also allow unused authorizations to be revoked.

Update: *Resolved in pull request #93. The OpenZeppelin Accounts team stated:*

> *Resolved in PR #93.*
>
> *We added `address executor` as the first field of both signed structures, hashed as `address(this)`:*
>
> ```
> keccak256("ScheduleRecover(address executor,bytes32 salt,bytes32 mode,bytes
> recoveryCalldata)")
> keccak256("CancelRecover(address executor,bytes32 salt,bytes32 mode,bytes
> recoveryCalldata)")
> ```
>
> *We kept the account's ERC-5267 domain rather than switching `verifyingContract` to the module, since the account domain is what lets a guardian's wallet display the account as the signing domain. Binding through a struct field preserves that while making an authorization valid for exactly one deployment.*
>
> *We extended this to `CANCEL_RECOVER_TYPEHASH` as well, per the finding's reference to the recovery and cancellation structures. Leaving cancellation unbound would keep a smaller replay open and leave the two typehashes asymmetric.*
>
> *Two regression tests install a second deployment of the module on the same account with the same guardians and assert the signature does not carry over: `testScheduleMultisignatureDoesNotReplayAcrossExecutors` and `testCancelMultisignatureDoesNotReplayAcrossExecutors`.*
>
> *We did not add a nonce or epoch. The module binding plus the existing operation-id uniqueness covers the replay path described here, and a nonce would add per-account state for a revocation capability that guardians can already get by canceling.*

L-03 ERC-5267 Domain Extensions Prevent Standard Guardian Recovery Signatures

The `_getTypedHash` function reads the account's ERC-5267 domain but ignores its `extensions` array and reconstructs a separator from only the five base fields. The module's `onInstall` function does not reject accounts that use unsupported domain extensions.

For an account with an extended EIP-712 domain, signatures produced from the reported domain do not match the module's base-only digest. Both `startRecovery` and guardian-

authorized `cancelRecovery` therefore reject otherwise valid extension-aware signatures. The bundled account implementation does not currently use extensions, so this affects other compatible account integrations.

Consider rejecting installation when `eip712Domain()` reports unsupported extensions, or reconstructing the complete domain when those extensions are supported.

Update: *Acknowledged, not resolved. The OpenZeppelin Accounts team stated:*

> *Ignoring the `extensions` array when reconstructing the domain is deliberate and consistent with how the library already treats this parameter.*
>
> *ERC-7739 states the position explicitly in its Rationale: "The `bytes1 fields` bitmap and `uint256[] extensions` array in ERC-5267 have been omitted. Differentiating between an absent field versus a zero field (e.g. `bytes32(0)`) offers no additional security benefits for on-chain defensive rehashing. The `extensions` parameter is a list of EIP numbers used for off-chain signaling."*
> *`ERC7739._isValidNestedTypedDataSignature` follows that and discards the `extensions` return value when rebuilding the app separator.*
> *`ERC7579SocialRecoveryExecutor._getTypedHash` applies the same rationale, and we would rather keep one consistent treatment across the library than have the social recovery module diverge.*
>
> *Two supporting points on the suggested remediations. Reconstructing the complete domain is not generically implementable: ERC-5267 defines `extensions` as a list of EIP numbers, each of which supplies its own field names, types, and a method to obtain their values. There is no common encoding to rebuild a separator from, and no finalized EIP currently defines an EIP-712 domain extension, so there is nothing concrete to implement against. Rejecting installation would turn an off-chain signaling value into an on-chain compatibility gate, which is the treatment ERC-7739 specifically moved away from.*
>
> *We also note the behavior is fail-closed: an extension-aware signature is rejected rather than wrongly accepted, so the consequence is a failed recovery attempt, not an unauthorized one.*

Notes & Additional Information

N-01 Inaccurate Documentation of `delayChangeWindow`

The `delayChangeWindow` function is documented as the minimum delay after which `setDelay` takes effect. In practice, it caps the transition period for delay increases, while decreases use the difference between the current and new delays. The value is also used as the default delay when none is provided during installation.

Consider documenting both uses of `delayChangeWindow` and clarifying that it is not a minimum transition period for every delay change.

Update: *Resolved in pull request #95. The OpenZeppelin Accounts team stated:*

> *Resolved in PR #95.*
>
> *The docstring now states both directions:*
>
> ```
> /**
> * @dev Maximum transition period applied when {setDelay} increases the delay,
> which takes effect
> * after `min(newDelay, delayChangeWindow())`. Delay decreases are unaffected and
> take effect after
> * `currentDelay - newDelay`. Also used as the default delay if not provided
> during {onInstall}.
> */
> ```
>
> *One note on scope: the second use was already documented accurately in the same docstring, since `_decodeDelayedExecutorInitData` falls back to `delayChangeWindow()` for `initData.length <= 7`. We kept that sentence and rewrote only the first, which was the inaccurate one. The contract-level docstring and the `setDelay` docstring already described the asymmetric behavior correctly, so this was an isolated inconsistency rather than a gap across the delay documentation.*

Conclusion

The audited changes introduce weighted guardian-based social recovery with delayed execution for ERC-7579 accounts. Overall, the implementation was found to be well structured and consistent with the surrounding OpenZeppelin Accounts architecture. The findings were limited to low-severity edge cases involving recovery authorization and account integration, together with a minor documentation issue.

The OpenZeppelin Accounts team is appreciated for its responsiveness and collaboration throughout the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.